



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS
DEPARTAMENTO DE ENGENHARIAS E TECNOLOGIA

PEN2049 - CÁLCULO NUMÉRICO

**PROJETO DE CÁLCULO NUMÉRICO - SOLUÇÃO PARA O JOGO LIGHTS OUT
RELATÓRIO TÉCNICO – LIGHTS OUT SOLVER**

MARIA LUMA DA SILVA OLIVEIRA
MATEUS GOMES SALOMÉ

Pau dos Ferros - RN

2026

1. INTRODUÇÃO

O desenvolvimento de métodos computacionais para resolução de problemas matemáticos constitui uma das principais aplicações da Álgebra Linear e do Cálculo Numérico. Diversos problemas encontrados em engenharia, computação, física, economia e outras áreas do conhecimento podem ser representados por sistemas lineares, cuja solução pode ser obtida por técnicas numéricas adequadas.

Entre os diversos exemplos que podem ser modelados matematicamente encontra-se o jogo Lights Out, um quebra-cabeça lógico criado na década de 1990 pela empresa Tiger Electronics. O jogo consiste em um tabuleiro formado por células que podem assumir dois estados possíveis: ligada (1) ou desligada (0). O objetivo do jogador é apagar todas as luzes do tabuleiro por meio de uma sequência adequada de pressionamentos sobre as células.

A principal característica do jogo é que cada pressionamento não altera apenas a célula selecionada, mas também determinadas células vizinhas. Dessa forma, cada jogada influencia simultaneamente várias posições do tabuleiro, tornando a resolução do problema mais complexa à medida que o tamanho do tabuleiro aumenta.

Do ponto de vista matemático, o problema pode ser modelado como um sistema linear definido sobre o corpo finito $GF(2)$, no qual todas as operações são realizadas módulo 2. Nessa representação, cada célula do tabuleiro corresponde a uma variável do sistema, enquanto os efeitos provocados por cada jogada são representados por uma matriz denominada matriz de influência.

A partir dessa modelagem, a resolução do jogo passa a ser equivalente à resolução de um sistema linear da forma: $Ax = b \pmod{2}$. Onde a matriz A representa os efeitos das jogadas sobre o tabuleiro, o vetor b representa o estado inicial das luzes e o vetor x representa as jogadas necessárias para apagar todas as luzes.

Neste trabalho foi desenvolvido o projeto Lights Out Solver, uma aplicação computacional capaz de construir automaticamente a matriz de influência, gerar tabuleiros manualmente ou de forma automática, resolver o sistema linear utilizando eliminação gaussiana módulo 2 e apresentar a solução encontrada juntamente com uma simulação passo a passo do processo de resolução.

Além da implementação em Python, foi desenvolvida uma versão em Julia, explorando diferentes abordagens para computação científica e permitindo uma comparação entre linguagens amplamente utilizadas em aplicações matemáticas e numéricas.

2. OBJETIVOS

Nesta seção são apresentados os objetivos que orientaram o desenvolvimento deste trabalho. Inicialmente é descrito o objetivo geral do projeto e, em seguida, são apresentados os objetivos específicos que definem as principais funcionalidades e conceitos abordados na implementação do solucionador do jogo Lights Out.

2.1 Objetivo Geral

Desenvolver uma aplicação computacional capaz de resolver automaticamente o jogo Lights Out por meio da modelagem matemática do problema como um sistema linear binário.

2.2 Objetivos Específicos

- Construir automaticamente a matriz de influência associada ao tabuleiro;
- Permitir a criação manual ou automática de tabuleiros;
- Implementar o método de eliminação gaussiana módulo 2;
- Aplicar busca de pivô, troca de linhas, eliminação e retro-substituição;
- Exibir a solução encontrada na forma de vetor e tabuleiro;
- Apresentar a resolução passo a passo;
- Validar a solução obtida;
- Permitir diferentes regras de vizinhança;
- Desenvolver implementações em Python e Julia;
- Explorar conceitos de posto, dependência linear, espaço nulo e existência de soluções.

3. DESENVOLVIMENTO

Nesta seção são apresentados os principais aspectos relacionados ao desenvolvimento do projeto, incluindo as linguagens utilizadas, a modelagem matemática empregada, o funcionamento do algoritmo de resolução e os componentes mais importantes da implementação. Também são descritos os procedimentos necessários para a execução do sistema e os conceitos teóricos aplicados na solução do problema.

3.1 Linguagens Utilizadas

Nesta subseção são apresentadas as linguagens de programação utilizadas no desenvolvimento do projeto.

3.1.1 Python

A linguagem Python foi escolhida devido à sua simplicidade sintática, ampla documentação e facilidade para implementação de algoritmos matemáticos. Sua estrutura clara permite que estudantes e pesquisadores compreendam facilmente a lógica utilizada na construção do sistema. Além disso, Python possui suporte robusto para manipulação de matrizes, estruturas de dados e aplicações científicas.

Após a instalação do Python, o programa pode ser executado através do terminal utilizando: `python lights_out_solver.py` ou `python3 lights_out_solver.py`.

Durante a execução, o usuário informa:

1. O tamanho do tabuleiro;
2. A regra de vizinhança;
3. O modo de geração do tabuleiro.

Após essas escolhas, o sistema realiza automaticamente a resolução do problema.

3.1.2 Julia

A linguagem Julia foi escolhida por ser uma linguagem moderna voltada para a computação científica. Uma das principais vantagens da Julia é seu elevado desempenho em operações matriciais, tornando-a adequada para aplicações envolvendo Álgebra Linear e métodos numéricos. Além disso, foi possível desenvolver uma interface gráfica utilizando a biblioteca Gtk.jl, permitindo uma interação mais intuitiva com o sistema.

Após a instalação da linguagem Julia e da biblioteca Gtk.jl, o programa pode ser executado através do comando: `julia lights_out_solver.jl`. A versão em Julia disponibiliza interface gráfica, permitindo a configuração do tabuleiro e visualização dos resultados de maneira interativa.

3.2 Modelagem Matemática do Problema

O estado de cada célula é representado por um valor binário:

0 = desligada

1 = ligada

Cada possível jogada também é representada por uma variável binária:

0 = não pressionar

1 = pressionar

Dessa forma, um tabuleiro $n \times n$ gera um sistema com n^2 variáveis. O problema é modelado pela equação: $Ax = b \pmod{2}$

onde:

- A é a matriz de influência;
- x é o vetor de jogadas;
- b é o vetor correspondente ao estado inicial do tabuleiro.

Todas as operações são realizadas módulo 2, pois os estados possíveis são apenas 0 e 1.

3.3 Construção da Matriz de Influência

A matriz de influência representa o efeito produzido por cada jogada. Cada coluna da matriz corresponde ao efeito de pressionar uma determinada célula. Por exemplo, considere um tabuleiro 3×3 .

Ao pressionar a célula central, são alteradas:

- a própria célula;
- a célula superior;
- a célula inferior;
- a célula à esquerda;
- a célula à direita.

Esses efeitos são registrados por valores iguais a 1 na coluna correspondente. O processo é repetido para todas as células do tabuleiro, gerando automaticamente a matriz A . Essa matriz constitui o núcleo matemático do problema.

3.4 Implementação da Eliminação Gaussiana

Para resolver o sistema linear foi implementado o método da eliminação gaussiana módulo 2. O algoritmo desenvolvido executa as seguintes etapas: Busca de pivô. Para cada

coluna procura-se uma linha contendo valor 1 que possa atuar como pivô. Essa etapa garante a continuidade do processo de escalonamento. Troca de linhas. Quando o pivô não está na posição desejada, realiza-se a troca de linhas. Essa operação é fundamental para evitar falhas durante o escalonamento. Eliminação. Após definir o pivô, as linhas inferiores são modificadas utilizando operações módulo 2.

A operação utilizada é: $\text{Linha}_j = \text{Linha}_j + \text{Linha}_i \pmod{2}$. Como as operações são realizadas módulo 2, a soma entre valores binários produz:

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 0$$

Essa característica equivale à operação lógica XOR.

Retro-substituição

Após o escalonamento, os valores das variáveis são obtidos de trás para frente. O resultado final corresponde ao vetor solução x .

3.5 Principais Componentes da Implementação

A implementação do projeto foi dividida em módulos responsáveis pela modelagem matemática do problema, resolução do sistema linear e interação com o usuário através da interface gráfica. Nesta seção são apresentados os principais componentes do código desenvolvido em Julia.

3.5.1 Conversão de Coordenadas

Para facilitar a construção da matriz de influência, foi criada uma função responsável por transformar coordenadas bidimensionais em índices lineares

```
function indice(i, j, n)
    return (i - 1) * n + j
end
```

Essa função permite relacionar cada posição do tabuleiro com uma posição correspondente nos vetores e matrizes utilizados pelo método algébrico.

Por exemplo, em um tabuleiro 3×3 :

$(1,1) \rightarrow 1$

$(1,2) \rightarrow 2$

$(1,3) \rightarrow 3$

$(2,1) \rightarrow 4$

...

$(3,3) \rightarrow 9$

3.5.2 Definição das Regras de Vizinhança

O programa permite três diferentes regras de vizinhança. A seleção é realizada pela função `obter_direcoes()`.

```
function obter_direcoes(tipo)
    if tipo == 1
        return [(0, 0), (-1, 0), (1, 0), (0, -1), (0, 1)]
    elseif tipo == 2
        return [(0, 0), (-1, -1), (-1, 1), (1, -1), (1, 1)]
    elseif tipo == 3
        return [
            (0, 0),
            (-1, 0), (1, 0), (0, -1), (0, 1),
            (-1, -1), (-1, 1), (1, -1), (1, 1)
        ]
    else
        error("Tipo de vizinhança inválido.")
    end
end
```

As opções disponíveis são:

- Ortogonal;
- Diagonal;
- Completa.

Essa funcionalidade atende ao requisito opcional de regras alternativas de vizinhança.

3.5.3 Construção da Matriz de Influência

A matriz de influência representa matematicamente o comportamento do jogo. Sua construção é realizada pela função `construir_matriz_influencia()`.

```
function construir_matriz_influencia(n, direcoes)
    tamanho = n * n
    A = zeros(Int, tamanho, tamanho)

    for i in 1:n
        for j in 1:n
            coluna = indice(i, j, n)

            # Verifica e aplica a influência nas posições vizinhas válidas
            for (di, dj) in direcoes
                ni = i + di
                nj = j + dj

                if 1 <= ni <= n && 1 <= nj <= n
                    linha = indice(ni, nj, n)
                    A[linha, coluna] = 1
                end
            end
        end
    end

    return A
end
```

Cada coluna representa uma jogada possível e cada linha representa uma luz do tabuleiro. A matriz gerada possui dimensão: $n^2 \times n^2$, e é utilizada para montar o sistema linear do problema.

3.5.4 Vetorização do Tabuleiro

Após definir o estado inicial do jogo, o tabuleiro é convertido em um vetor coluna por meio da função `vetorizar_tabuleiro()`.

```
function vetorizar_tabuleiro(tabuleiro)
    n = size(tabuleiro, 1)
    vetor = Int[]

    for i in 1:n
        for j in 1:n
            push!(vetor, tabuleiro[i, j] % 2)
        end
    end

    return vetor
end
```

Esse vetor corresponde ao termo independente do sistema linear: $Ax = b$

3.5.5 Eliminação Gaussiana Módulo 2

O núcleo matemático do projeto encontra-se na função `eliminacao_gaussiana_mod2()`, responsável por resolver o sistema linear utilizando operações em módulo 2. Inicialmente é criada a matriz aumentada:

```
matriz = hcat(copy(A), b .% 2)
```

Em seguida é realizada a busca de pivô:

```

for linha in linha_pivo:linhas
    if matriz[linha, coluna] == 1
        pivo = linha
        break
    end
end
end

```

Quando necessário ocorre a troca de linhas:

```

temp = copy(matriz[linha_pivo, :])
matriz[linha_pivo, :] = matriz[pivo, :]
matriz[pivo, :] = temp

```

Por fim, é executada a eliminação:

```

matriz[linha, k] =
    (matriz[linha, k] + matriz[linha_pivo, k]) % 2

```

Todas as operações são realizadas no módulo 2, conforme exigido na atividade.

3.5.6 Retro-substituição

Após o escalonamento, a solução é obtida por substituição para trás.

```

for i in length(colunas_pivo):-1:1
    coluna_pivo = colunas_pivo[i]
    soma = 0

    for coluna in (coluna_pivo + 1):colunas
        if coluna <= colunas
            soma = (
                soma + matriz[i, coluna] * solucao[coluna]
            ) % 2
        end
    end

    solucao[coluna_pivo] = (
        matriz[i, colunas + 1] + soma
    ) % 2
end

```

O resultado é um vetor binário indicando quais posições devem ser pressionadas.

3.5.7 Aplicação das Jogadas

Após encontrar a solução, o programa simula todas as jogadas encontradas. A função `aplicar_jogada!()` é responsável por atualizar o estado do tabuleiro.

```

for (di, dj) in direcoes
    ni = i + di
    nj = j + dj

    if 1 <= ni <= n && 1 <= nj <= n
        tabuleiro[ni, nj] = (tabuleiro[ni, nj] + 1) % 2
    end
end

```

Essa etapa possibilita mostrar ao usuário toda a resolução passo a passo.

3.5.8 Geração Automática de Tabuleiros Solucionáveis

O programa também possui um mecanismo para gerar tabuleiros que garantidamente possuem solução.

```
function gerar_tabuleiro_com_solucao(n, tipo_vizinhanca)
    tentativas = 0

    while true
        tentativas += 1
        tabuleiro = gerar_tabuleiro_automatico(n)

        # Valida se o tabuleiro gerado possui solução
        _, _, solucao, _, _ =
            resolver_tabuleiro(tabuleiro, tipo_vizinhanca)

        if solucao != nothing
            return tabuleiro, tentativas
        end
    end
end
```

Esse recurso evita que o usuário receba configurações impossíveis de resolver.

3.5.9 Interface Gráfica

A interação com o usuário foi implementada utilizando a biblioteca Gtk.jl. A interface permite criar tabuleiros, selecionar regras de vizinhança e executar a resolução automaticamente.

```
function criar_interface()
    # Define a Janela Principal
    janela = GtkWindow(
        "Solucionador Lights Out - Julia",
        1200,
        900
    )
```

A interface contém:

- seleção do tamanho do tabuleiro;
- seleção da regra de vizinhança;
- geração automática;
- criação manual do tabuleiro;
- botão de resolução;
- exibição completa dos resultados.

3.5.10 Geração do Relatório de Saída

Após a resolução, o sistema gera automaticamente um relatório contendo todas as informações relevantes do processo.

O relatório inclui:

- tabuleiro inicial;
- matriz de influência;
- vetor b;
- matriz escalonada;
- posto da matriz;
- vetor solução;
- mapa de jogadas;
- resolução passo a passo;
- tabuleiro final;
- validação da solução.

Além disso, o conteúdo é exportado automaticamente para o arquivo: resultado_lights_out.txt, permitindo o armazenamento permanente dos resultados obtidos.

4. DISCUSSÃO FINAL

Nesta seção são discutidos os resultados obtidos a partir da modelagem matemática do jogo Lights Out, relacionando-os com conceitos fundamentais da Álgebra Linear. São analisados aspectos como posto da matriz, dependência linear, espaço nulo e existência de soluções, destacando sua influência no processo de resolução do problema e na interpretação dos resultados encontrados.

4.1 Posto da Matriz

O posto de uma matriz corresponde ao número máximo de linhas ou colunas linearmente independentes. Durante o escalonamento, o posto é determinado pela quantidade de pivôs encontrados. No jogo Lights Out, o posto indica quantas informações independentes estão presentes no sistema. Quanto maior o posto, maior a independência entre as jogadas.

4.2 Dependência Linear

A dependência linear ocorre quando uma coluna pode ser escrita como combinação linear de outras colunas. No contexto do jogo, isso significa que determinadas jogadas produzem efeitos equivalentes sobre o tabuleiro. Assim, algumas jogadas tornam-se redundantes. A presença de dependência linear reduz o posto da matriz.

4.3 Espaço Nulo

O espaço nulo é definido pelo conjunto de soluções da equação: $Ax = 0$

No jogo Lights Out, os vetores pertencentes ao espaço nulo representam combinações de jogadas que não alteram o estado final do tabuleiro.

A dimensão do espaço nulo é dada por: $\dim(N(A)) = n - \text{posto}(A)$. Quanto maior o espaço nulo, maior a quantidade de soluções possíveis.

4.4 Existência de Soluções

A existência de solução depende da relação entre o posto da matriz dos coeficientes e o posto da matriz aumentada.

Segundo o Teorema de Rouché-Capelli:

- Se $\text{posto}(A) = \text{posto}([A|b])$, existe solução;
- Se $\text{posto}(A) \neq \text{posto}([A|b])$, não existe solução.

Esse comportamento foi observado durante os testes realizados no projeto. Determinados tabuleiros gerados aleatoriamente não possuíam solução para algumas regras de vizinhança.

4.5 Unicidade das Soluções

Durante a execução do projeto, a eliminação gaussiana módulo 2 permitiu identificar se o sistema linear associado ao tabuleiro possuía uma única solução ou múltiplas soluções. Quando o posto da matriz de influência é igual ao número de variáveis do sistema, todas as variáveis possuem pivôs e a solução encontrada é única. Nessa situação, existe apenas uma combinação de jogadas capaz de apagar todas as luzes do tabuleiro.

Por outro lado, quando o posto da matriz é inferior ao número de variáveis, surgem variáveis livres durante o processo de escalonamento. Isso significa que existem diferentes combinações de jogadas que satisfazem o sistema linear. No contexto do Lights Out, essas diferentes soluções representam sequências distintas de pressionamentos que conduzem ao mesmo resultado final: um tabuleiro completamente apagado.

No projeto desenvolvido, o algoritmo foi implementado para encontrar uma solução válida sempre que ela existe. Entretanto, em alguns tabuleiros, especialmente quando são utilizadas diferentes regras de vizinhança, podem existir várias soluções possíveis devido à presença de variáveis livres associadas ao espaço nulo da matriz de influência.

4.6 Relação entre os Conceitos

Os conceitos de posto, dependência linear, espaço nulo e existência de soluções mostraram-se diretamente relacionados durante a implementação do solucionador do jogo Lights Out.

A matriz de influência construída pelo programa descreve matematicamente como cada jogada afeta o estado do tabuleiro. Quando algumas colunas desta matriz podem ser obtidas como combinação linear de outras, ocorre dependência linear entre as jogadas. Essa

dependência reduz o posto da matriz, pois diminui a quantidade de informações independentes presentes no sistema.

A redução do posto implica um aumento da dimensão do espaço nulo. Em termos práticos, isso significa que existem vetores não nulos que, quando aplicados ao sistema, não alteram o estado final do tabuleiro. Consequentemente, diferentes vetores de jogadas podem produzir exatamente o mesmo resultado, gerando múltiplas soluções para o problema.

Além disso, a comparação entre o posto da matriz de influência e o posto da matriz aumentada, realizada durante a eliminação gaussiana, permite determinar a existência de solução. Quando ambos os postos são iguais, o sistema é consistente e o tabuleiro pode ser resolvido. Caso contrário, o sistema torna-se inconsistente e não existe sequência de jogadas capaz de apagar todas as luzes.

No projeto, esses conceitos não foram apenas estudados teoricamente, mas utilizados diretamente pelo algoritmo para construir a matriz de influência, escalonar o sistema, identificar pivôs, determinar o posto da matriz e encontrar uma solução válida. Dessa forma, o jogo Lights Out serviu como uma aplicação prática dos principais conceitos de Álgebra Linear e Cálculo Numérico, demonstrando como ferramentas matemáticas podem ser utilizadas para resolver problemas computacionais reais.

5. CONSIDERAÇÕES FINAIS

O desenvolvimento do projeto Lights Out Solver permitiu aplicar conceitos fundamentais de Álgebra Linear, sistemas lineares e métodos numéricos em um problema prático e visualmente intuitivo. A modelagem do jogo por meio de sistemas lineares mostrou como problemas aparentemente simples podem ser analisados matematicamente e resolvidos através de algoritmos computacionais eficientes.

A implementação da eliminação gaussiana módulo 2 possibilitou a obtenção automática das soluções, enquanto a construção da matriz de influência permitiu representar formalmente o comportamento do tabuleiro. Além dos aspectos computacionais, o projeto proporcionou uma compreensão mais profunda dos conceitos de posto, dependência linear, espaço nulo e existência de soluções, evidenciando sua aplicação prática na resolução de problemas.

Os resultados obtidos demonstram que a combinação entre teoria matemática e programação constitui uma ferramenta poderosa para a construção de soluções computacionais, contribuindo significativamente para a formação acadêmica em áreas relacionadas à Matemática Aplicada, Computação e Engenharia.

